World Models in Action

3. Representation learning before generation

This chapter covers

- The assumptions built into self-supervised objectives
- Contrastive, bootstrap, self-distillation and redundancy-reduction objectives
- Masked prediction and the choice of prediction target
- Latent bottlenecks and the limits of representation evaluation

A world model needs a representation that preserves distinctions relevant to predicting future observations. Pixels contain both useful information and details that a predictive model may not need. An encoder maps the observation x to a representation $z = f_\theta(x)$, but compression alone does not make z useful. Two observations that should lead to different predictions must not be merged merely because their images are similar.

When labels are unavailable, the training signal must be constructed from the data. Methods may identify a matching view among candidates, predict a representation produced by another branch, constrain feature statistics across a batch, or predict content hidden from the encoder. These objectives differ in their targets, their assumptions and their optimization dynamics.

The central question is therefore not just whether the training loss decreases. It is what the loss rewards the encoder for preserving. A feature may solve the pretraining task yet omit information needed for a later prediction. The following sections derive the principal objectives and explain exactly what their mathematical constraints do and do not establish.

3.1. Self-supervised learning foundations

> **The central distinction**
>
> Self-supervised learning constructs a relationship from data and optimizes an encoder to satisfy it. That relationship may be useful without being sufficient for a world model. Separate the training target, the learned representation and evidence about downstream state quality.

3.1.1. A representation is not just a compressed observation

Let x be an observation and let f_θ be an encoder with parameters θ. Its output is a vector z:

$$z = f_\theta(x).$$

A smaller representation is useful only if the information removed is unnecessary for the task. A vector may preserve object identity while losing location or orientation. It may describe a frame's appearance while losing motion that is apparent only from a sequence. Compression can remove nuisance detail, but it can also merge observations whose futures differ.

Chapter 2 formalized the desired contract in terms of sufficiency, minimality and aliasing. Self-supervised objectives do not observe that contract directly. They rely on a proxy relationship in the data. A representation may perform well on the proxy task while failing to retain information the world model needs.

3.1.2. Invariance is a decision

Suppose T_1 and T_2 are random transformations applied to the same input:

$$x_1 = T_1(x), \qquad x_2 = T_2(x).$$

A view-consistency objective encourages $f_\theta(x_1)$ and $f_\theta(x_2)$ to agree. The design decision is hidden in T_1 and T_2: they define which input differences the model is encouraged to ignore.

Random crops are useful when the cropped-out region is irrelevant to the intended representation. They are harmful when the missing region contains information needed to distinguish the scene or predict its next state. Color jitter can suppress nuisance lighting changes, but it can also remove a visual cue on which a later prediction depends. The transformation is part of the learning problem, not a neutral preprocessing step.

An ideal invariance would identify observations that the downstream task truly treats as equivalent. A practical augmentation only approximates that relation. We should not assume that two views are interchangeable for every task just because they came from the same image.

The distinction can be written directly into the objective. Let T_1, T_2 be random transformations sampled from a chosen distribution and let d measure disagreement between two features. A consistency objective has the form

$$L_{\text{cons}}(\theta) = \mathbb{E}_{x,\, T_1,\, T_2}\big[d(f_\theta(T_1 x),\, f_\theta(T_2 x))\big].$$

The expectation averages over the views the augmentation pipeline actually creates, not every transformation that might occur in the world. If a transformation removes information, the encoder is rewarded for making the two resulting views agree despite that missing information. Information that varies unpredictably across views cannot reliably help reduce this loss. This is how augmentation defines an implicit preference over what the representation keeps.

A useful view changes nuisance factors while leaving the intended shared content recoverable. A crop may preserve object identity but alter visible context; a color transformation may preserve shape but alter appearance. That assumption belongs to the data and task, not to the loss function

itself.

This can be made precise. Let $\mathcal{T}$ be a set of transformations and define the orbit of an input x as $\mathcal{O}(x) = \{T(x) : T \in \mathcal{T}\}$. If an encoder is exactly invariant, then

$$f_\theta(Tx) = f_\theta(x) \qquad \text{for every} T \in \mathcal{T}.$$

All observations in the same orbit receive the same representation. The encoder therefore acts on equivalence classes of inputs rather than on every pixel-level distinction. This is the mathematical meaning of invariance: it removes distinctions that the chosen transformation family treats as irrelevant.

That reduction is beneficial only if the target of interest is constant within each class. If Y changes when x is transformed, no exactly invariant representation can preserve all information about Y. The problem is not that optimization failed; the objective has asked the encoder to discard the distinction. In real systems the equality is approximate, but the design pressure remains.

Invariance is also different from **equivariance**. An invariant representation remains unchanged under a transformation. An equivariant representation changes in a predictable way. For example, if an image is translated, an image-level category feature may ideally remain fixed, while a spatial feature map may shift in correspondence with the image. Classification and localization therefore need not prefer the same transformation behavior. A feature map that is deliberately equivariant can be more useful than a globally invariant vector when the target depends on where a feature occurs.

Historically, visual self-supervision used hand-designed pretext tasks such as predicting image rotation (Gidaris et al., 2018), restoring the spatial order of jigsaw patches (Noroozi and Favaro, 2016), predicting spatial context (Doersch et al., 2015), and colorizing grayscale images (Zhang et al., 2016). These tasks assign synthetic targets to unlabeled data and make the target-design assumption easy to see. They remain useful conceptual examples, but the architecture survey below focuses on the broader contrastive, bootstrap, statistical, clustering and masked-prediction families that followed.

These earlier tasks are often called pretext tasks: labels are created automatically from the input, and the encoder is trained to solve the constructed task. Their weakness is not that they fail to learn features. It is that the pretext label may be predictable from shortcuts or may require only a subset of the information needed elsewhere. An encoder can predict rotation using scene-layout cues without learning a representation that preserves every detail of the scene. This motivates objectives whose targets arise from relationships between views, statistics of feature batches, or missing content rather than one fixed synthetic label.

The principle behind representation learning

Every self-supervised method constructs a target relationship from the data. The relationship may be agreement between views, a ranking among candidates, a teacher's output distribution, statistics of a feature batch, a cluster assignment or a missing patch. Ask what information makes that relationship easy to satisfy, then ask whether that information is what the downstream model needs.

The map below groups methods by their dominant training signal while allowing overlap between families.

The family map in figure 3.1 groups objectives by the signal they construct, rather than by publication date.

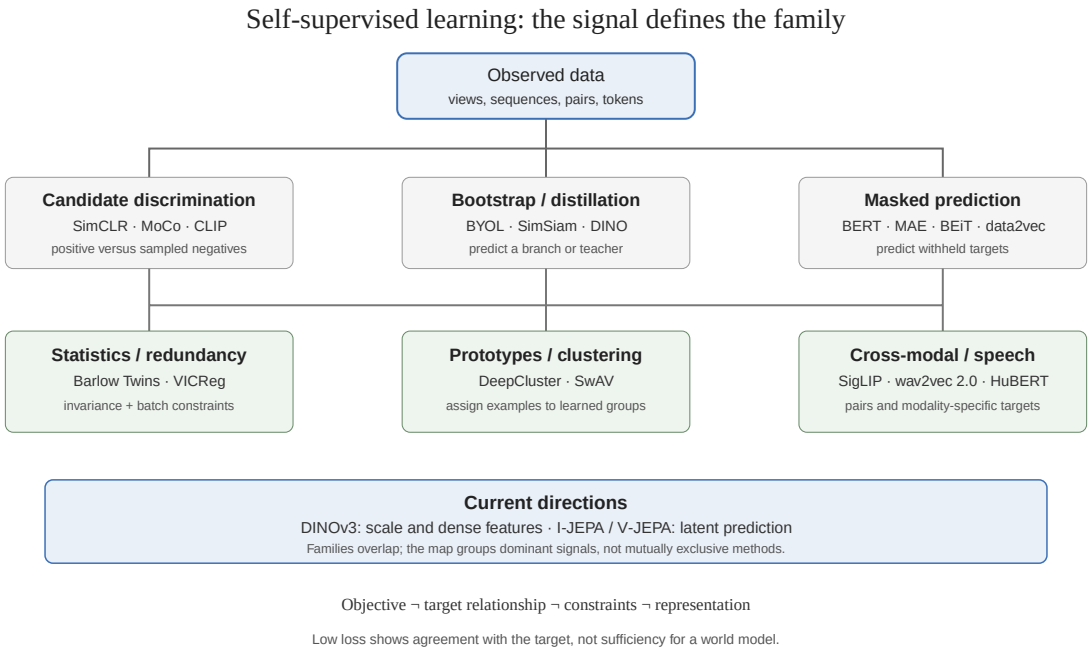


Figure 3.1 Self-supervised learning families, grouped by their training signal

The map is not a partition into disjoint categories. DINO uses teacher-student self-distillation over prototype outputs; iBOT joins masking and teacher targets; wav2vec 2.0 combines masking, quantization and contrastive discrimination.

3.2. Contrastive learning

3.2.1. From pairs to candidate classification

A contrastive method begins with an anchor, a related positive example and distractors called negatives. The encoder maps each example to a feature vector. A similarity function scores the anchor against each candidate. Training rewards the positive for receiving a higher score than the negatives.

In the common two-view construction, each source example x_n is transformed independently twice:

$$\tilde{x}_n^{(1)} = T_n^{(1)}(x_n), \qquad \tilde{x}_n^{(2)} = T_n^{(2)}(x_n).$$

The two views of the same source form a positive pair. Views from other source examples form the negative candidates for that anchor. No class label is required; source identity supplies the pair label. The label therefore means “same source under two transformations,” not necessarily “same semantic class” or “same underlying state.”

If two different source examples are semantically equivalent, treating one as another’s negative creates a false negative relative to a semantic objective. This is not an error in cross-entropy. It is a mismatch between the labels implied by pair construction and the relation we ultimately want the representation to encode.

The negative set is part of the statistical problem, not just a source of extra examples. A contrastive loss trains the encoder to preserve distinctions that help identify the positive against the sampled candidates. If negatives are easy, the model can often solve the task using broad visual cues. As training improves, the most probable negatives dominate the gradient and the model is pushed to distinguish harder cases. This can sharpen useful features, but it can also make the learned geometry depend on details of the candidate distribution.

The loss does not tell the model which differences are semantically important. It only supplies a ranking label for the candidates. If the dataset repeatedly pairs two views of the same source image while treating every other image as negative, the representation is encouraged to keep same-source views close and other sampled views relatively separated. Two different images of the same object may still be pushed apart. Methods that use multiple positives, debiased negatives or semantic labels change this constraint, but they also change the learning problem.

For two feature vectors, cosine similarity is their dot product after unit normalization:

$$s(z_i, z_j) = \frac{z_i^\top z_j}{\| z_i \|_2 \, \| z_j \|_2}.$$

A temperature $\tau>0$ scales every similarity before softmax. If two candidates have similar scores, a lower temperature magnifies their difference in the logits and makes the probability distribution

more concentrated. It does not simply make every gradient larger: when the positive already has probability close to one, its cross-entropy gradient is close to zero. Temperature changes both the probability assigned to each candidate and the factor relating a logit gradient to the underlying similarity gradient.

Suppose a batch contains N source images. Two stochastic views of each image give $2N$ encoded vectors. For anchor i, let $p(i)$ be the index of its other view. Excluding the anchor itself, every remaining view is a candidate. The NT-Xent loss for that anchor is

$$\ell_i = -\log \frac{\exp(s(z_i, z_{p(i)}) / \tau)}{\sum_{a=1}^{2N} \mathbf{1}[a \neq i] \exp(s(z_i, z_a) / \tau)}.$$

The batch loss averages this quantity over all $2N$ anchors:

[$$L_{\mathrm{NT\text{-}Xent}}=\frac{1}{2N}\sum_{i=1}^{2N}\ell_i.$$] | *stem-a16c682eaf9b33914e71c1e010c03848.svg*

In the denominator of each term are one positive and $2N-2$ negatives. The positive is included in the denominator; the anchor is not. Stating the candidate set explicitly avoids ambiguity because papers do not all use the same symbol for the number of candidates or negatives.

This objective is ordinary softmax cross-entropy. Write $a_{ij} = s(z_i, z_j) / \tau$, and let q_{ij} be the softmax probability assigned to candidate j. Differentiating with respect to each candidate logit gives

$$\frac{\partial \ell_i}{\partial a_{ij}} = q_{ij} - \mathbf{1}[j = p(i)].$$

The positive logit receives gradient $q_{i,\,p(i)} - 1$, which pushes it upward under gradient descent. A negative logit receives gradient q_{ij}, which pushes it downward. The gradient on a negative is weighted by its current probability: a hard negative receives more repulsive gradient than an already dissimilar candidate. Since $a_{ij} = s(z_i, z_j) / \tau$, differentiation with respect to the unscaled similarity introduces the factor $1 / \tau$.

For example, suppose one anchor has cosine similarities 0.80, 0.10 and -0.30 to its positive and two negatives, with $\tau = 0.5$. The logits are 1.6, 0.2 and -0.6. Softmax assigns the positive probability about 0.737, so the loss for this anchor is $-\log(0.737) \approx 0.306$. If all three candidates received equal probability, the loss would be $-\log(1/3) \approx 1.099$. The loss measures whether the correct candidate is easy to identify relative to the candidates that happened to be sampled.

The logit derivative explains the direction of the update. If the positive has probability q_+, its derivative is $q_+ - 1$, which is nonpositive. Gradient descent therefore raises the positive logit unless it is already assigned probability one. A negative with probability q_j has positive derivative q_j, so its logit is lowered. The gradient focuses on candidates that the model currently finds plausible;

easy negatives contribute little because their probabilities are already small.

For a cosine score $s(u, v) = \hat{u}^\top \hat{v}$, where $\hat{u} = u / \| u \|_2$ and $\hat{v} = v / \| v \|_2$, the derivative with respect to the first vector is

$$\frac{\partial s(u, v)}{\partial u} = \frac{1}{\| u \|_2}(\hat{v} - s(u, v)\hat{u}).$$

This gradient changes the direction of a normalized vector to improve its relative score. In a neural network, the same representation can appear as an anchor and as a candidate in other loss terms, so its total gradient accumulates contributions from all those roles. Contrastive learning is not merely "pull positives together": it adjusts the representation so that positives outrank the negatives in the sampled candidate set.

Figure 3.2 visualizes the anchor, candidate set, softmax distribution and the resulting gradient.

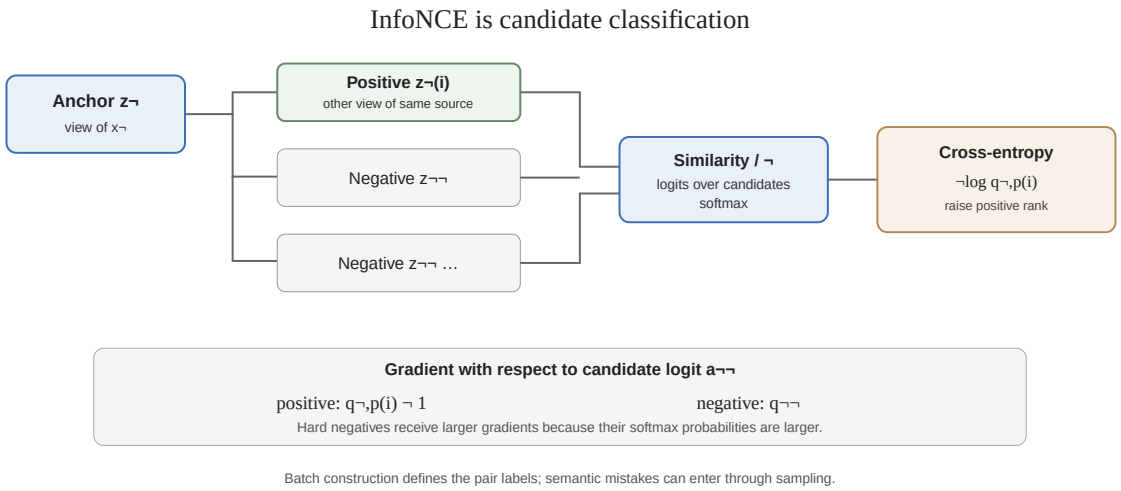


Figure 3.2 InfoNCE turns a similarity comparison into candidate classification

3.2.2. What InfoNCE does and does not prove

InfoNCE is a relative objective. The anchor must assign a higher score to its designated positive than to the sampled alternatives. Maximizing the positive similarity alone would not impose this ranking constraint. With an unnormalized dot product the model can also increase scores by increasing vector norms; cosine normalization bounds the score but does not remove dependence on the candidate-sampling protocol.

The positive label is supplied by data construction. In a two-view image objective, it means that two views came from the same source image. A negative can still depict the same semantic class as the anchor or show a nearly identical state. This is a false negative relative to a semantic objective, not an error in the cross-entropy arithmetic. The transformation distribution creates a

related risk: if a view removes a relevant feature, consistency across those views can reward the encoder for ignoring it.

InfoNCE also has an information-theoretic interpretation, but only under a specified sampling construction. Suppose one index i is selected uniformly from K candidates; y_i is sampled from the conditional distribution $p(y \mid x)$, while the other $K-1$ candidates are sampled independently from the marginal $p(y)$. Bayes' rule gives

$$P(i \mid x, y_1, \ldots, y_K) = \frac{r(x, y_i)}{\sum_{j=1}^{K} r(x, y_j)}, \qquad r(x, y) = \frac{p(y \mid x)}{p(y)}.$$

The optimal classifier therefore ranks candidates by the density ratio: a pair receives a high score when it is more likely under the joint distribution than under independent sampling. A learned critic approximates this rule through its logits. The candidate-classification cross-entropy under this sampling construction yields the lower bound

$$I(X; Y) \geq \log K - \mathcal{L}_{\mathrm{NCE}},$$

where K includes the positive candidate. Equivalently, $\mathcal{L}_{\mathrm{NCE}} \geq \log K - I(X; Y)$. The finite candidate count limits this bound to $\log K$. The bound does not grow without limit merely because training continues.

The assumptions matter. Nearby frames, repeated examples or multiple instances of one category can violate independent negative sampling. If negatives come from a different distribution, or candidates are dependent, the same bound does not automatically follow. The density-ratio argument explains the role of the loss and its sampling assumptions; it does not turn an arbitrary contrastive loss into a certified measure of semantic information.

Finally, mutual information shared by two views is not the same as sufficiency for a later prediction. A feature can be shared between views yet irrelevant to the future quantity of interest. The information-theoretic bound says something about dependence under the sampling model, not whether all distinctions required by a downstream predictor have been retained.

3.2.3. SimCLR and MoCo: two ways to construct candidates

SimCLR (Chen et al., 2020) made the augmented-view recipe explicit. Two views pass through an encoder f and a nonlinear projection head g, and the objective is applied to $z = g(f(x))$. The representation before the head, $h = f(x)$, is commonly retained for downstream tasks. The projector allows the loss to shape a space specialized for the pretraining comparison, rather than requiring every property of that space to be optimal for later evaluation. This is a design choice supported by the SimCLR experiments, not a theorem that a projection head always improves every representation.

SimCLR's results depended on augmentation composition, batch size, architecture, training duration and optimization. A large batch increases the number of available negatives under an in-batch objective, but changes the classification problem itself and may add more false negatives. Its contribution is a clear recipe for contrastive visual learning, not evidence that every contrastive method needs one specific batch size or projector.

MoCo (He et al., 2020) uses a query encoder and a key encoder, with a queue storing keys from earlier minibatches. This gives each query a larger candidate set than the current batch alone. The key encoder follows an exponential moving average:

$$\theta_k \leftarrow m\theta_k + (1-m)\theta_q.$$

The queue contains keys produced by earlier parameter values. They become stale as training proceeds; a slowly moving key encoder reduces abrupt drift in their representation space but cannot make historical keys identical to current ones. The update [stem 3484bd6dc221aec4b5bf81c08b08450f] makes the key parameters a smoothed history of the query parameters: for fixed m, contributions from earlier updates decay geometrically. MoCo v2 (Chen et al., 2020) changed the training recipe, while MoCo v3 (Chen et al., 2021) studied a different recipe for vision transformers.

Both methods remain sensitive to the definition of positive pairs and negatives. A queue solves the size and update-consistency problem; it does not establish that every negative is semantically different.

3.2.4. CPC as a historical bridge

Contrastive Predictive Coding (CPC; van den Oord et al., 2018) encodes a sequence and uses a context vector from the past to score future latent representations. An InfoNCE-style objective teaches the context to identify the observed future latent among candidate latents. CPC is a useful historical bridge between contrastive discrimination and predictive representations, but it is not the centerpiece of a contemporary visual SSL survey.

A CPC future-latent retrieval score only shows that the correct future code can be identified among the sampled candidates under the chosen objective. It does not by itself establish accurate multi-step rollout, a sufficient predictive state, or that errors remain controlled when predictions are fed back into the model.

3.3. Non-contrastive objectives

Contrastive learning makes a designated positive score higher than sampled negatives. Non-contrastive methods remove that explicit competition, but they still need a mechanism that prevents agreement from becoming trivial. BYOL and SimSiam use asymmetric prediction

pathways; DINO matches teacher distributions; Barlow Twins and VICReg constrain batch statistics.

3.3.1. Why agreement alone admits collapse

Suppose two views of the same input are trained only to produce equal representations. A simple invariance loss is

$$L_{\text{inv}} = \frac{1}{N}\sum_{i=1}^{N} \| z_i - z_i' \|_2^2 .$$

If the encoder maps every input to the same constant vector c, then $z_i = z_i' = c$ for every example and the loss is zero. The objective has no term requiring different inputs to remain distinguishable. A constant output is therefore a collapsed solution, even though it is useless for representing the data.

This observation does not mean every method that matches two views will collapse. It means agreement alone cannot rule collapse out. The architecture, stop-gradient placement, target updates, normalizations and optimization dynamics affect which solutions training reaches. The methods below differ in precisely those mechanisms; none should be described as having a universal guarantee based only on its loss formula.

It is useful to distinguish complete collapse from dimensional collapse. In complete collapse, all inputs map to the same vector and the representation contains no information about differences between examples. In dimensional collapse, the output varies, but most of its covariance is concentrated in a small number of directions. A model can therefore avoid the constant solution and still use its nominal d-dimensional space inefficiently.

These cases require different diagnostics. Per-coordinate variance detects a constant dimension; the spectrum of the covariance matrix shows whether variation is spread across many directions. But neither diagnostic answers whether those directions contain the information of interest. A feature can vary strongly with background texture and have high effective rank while carrying little information about the target. Anti-collapse is necessary for useful representations, not sufficient.

3.3.2. BYOL and SimSiam: bootstrap prediction

BYOL (Grill et al., 2020) has an online encoder, a projection head and a predictor, alongside a target encoder and target projection head. Only the online network receives gradients. The target network is updated by an exponential moving average (EMA) of the online encoder and projector; it is not updated by backpropagation.

Let $z_1 = g_\phi(f_\theta(x_1))$, $p_1 = q_\psi(z_1)$, and $\bar{z}_2 = g_{\bar{\phi}}(f_{\bar{\theta}}(x_2))$. The prediction loss for one direction is the

normalized regression

$$D(p_1, (\bar{z}_2)) = 2 - 2\frac{p_1^\top \bar{z}_2}{\| p_1 \|_2 \, \| \bar{z}_2 \|_2},$$

where [stem b612eeef4ccf6658aa1eb7c15510a140] denotes stop-gradient. A second term swaps the views. The online predictor is trained to match the target feature, while the target feature is treated as fixed for that gradient calculation.

The target parameters change according to

$$\bar{\theta} \leftarrow m\bar{\theta} + (1-m)\theta, \qquad \bar{\phi} \leftarrow m\bar{\phi} + (1-m)\phi,$$

with m close to one. The target is therefore a smoothed history of the online network, not an independently optimized network. This is the defining difference from ordinary shared-weight Siamese regression.

With fixed momentum m, repeated EMA updates give old parameters geometrically decreasing influence. The target is consequently neither fixed nor an exact copy of the current online network: it is a moving reference that changes more slowly than the optimized branch. This temporal asymmetry is an essential part of BYOL's method, not merely a way to save computation.

For clarity, the two-view objective can be written

$$L_{\mathrm{BYOL}} = D(p_1, (\bar{z}_2)) + D(p_2, (\bar{z}_1)).$$

During a gradient step, the first term updates the online encoder, projector and predictor through p_1; it does not update the target parameters through $\bar{z}_2$. The second term reverses the roles of the views. After the gradient update, the EMA rule updates the target parameters. Thus two distinct operations occur: gradient descent optimizes the online network against a fixed target value for that step, and parameter averaging moves the target afterward.

This order explains why an EMA target is more than a detached copy. A detached copy would stop gradients through the target but would not specify how the target changes over training. EMA defines a low-pass update: fast fluctuations in online weights have less immediate effect on the target, while persistent changes gradually pass into it. In the limiting case $m = 0$, the target copies the online weights after each update; as m approaches one, the target evolves more slowly. This describes the update mechanism, not a universal proof against collapse.

SimSiam (Chen and He, 2021) uses the same encoder and projector for both views and a predictor on each branch, but no EMA target network. With $z_i = g(f(x_i))$ and $p_i = q(z_i)$, its symmetric loss has the form

$$L_{\mathrm{SimSiam}} = D_{\mathrm{SimSiam}}(p_1, (z_2)) + D_{\mathrm{SimSiam}}(p_2, (z_1)),$$

where D_{SimSiam} is negative cosine similarity. The stop-gradient leaves a target's forward value unchanged but removes the gradient path through that target in the corresponding term. Because the loss is symmetric, each shared branch acts as predictor in one term and target in the other. BYOL instead maintains a separate target network whose parameters follow the EMA update.

The predictor and stop-gradient change the optimization dynamics; they do not remove the constant-output solution from the mathematical objective.

The distance used in the symmetric objective can be written

$$D_{\text{SimSiam}}(p, z) = -\frac{p^\top z}{\| p \|_2 \, \| z \|_2}.$$

Minimizing this negative cosine similarity maximizes directional agreement without requiring the two vectors to have the same norm. In $D(p_1, (z_2))$, the numerical value of z_2 still affects the loss, but its gradient path is removed. The shared encoder can nevertheless receive gradients through z_2 when that representation appears as the predictor-side output in the opposite term. Stop-gradient is applied to a particular **role** of the representation in each term, not as a permanent property of the tensor.

This distinction matters when interpreting a computational graph. "The target branch has no gradient" is accurate for a single detached target path; it would be inaccurate to say that the shared encoder receives no gradient from the second view across the full symmetric loss. The symmetry of the loss and the asymmetry of each gradient path coexist.

```
The original SimSiam study found that stop-gradient was essential to its
proposed configuration. That is a method-specific result, not a theorem that any
Siamese network with a detached branch must learn non-collapsed features.
```

The predictor makes the two sides of the training relation structurally asymmetric. The online path must learn a transformation q whose output matches the other branch's representation, while the target path's value is held fixed for that loss term. If the target also received gradients from the same term, the two sides could reduce the discrepancy by moving together immediately. Stop-gradient prevents that direct route for the target side, but it does not by itself force informative features. The predictor, normalization, shared parameters and optimization dynamics together determine whether training moves away from collapse in the original recipes.

3.3.3. DINO: match a teacher's distribution

DINO (Caron et al., 2021) matches distributions rather than regressing one feature vector onto another. The student and teacher map different views to logits over learned prototype outputs. If $a_s(x)$ and $a_t(x)$ are their logits, the student and teacher distributions are

$$p_s(x) = \left(\frac{a_s(x)}{T_s} \right), \qquad q_t(x) = \left(\frac{a_t(x) - c}{T_t} \right).$$

Here c is a running center of teacher outputs, T_t controls teacher-target sharpness, and the teacher parameters follow an EMA of the student parameters. The student is trained across selected, different teacher-student views. For a target distribution q_t, the student's cross-entropy is

$$H(q_t, p_s) = -\sum_{k=1}^{K} q_{t,k} \log p_{s,k}.$$

Training sums this loss over selected teacher-student view pairs. Centering subtracts a running output mean before the teacher softmax; sharpening uses a low teacher temperature to lower target entropy. They serve different roles: centering reduces persistent preference for a subset of output coordinates, while sharpening produces a more decisive target distribution.

Cross-entropy decomposes into the entropy of the target plus a divergence:

$$H(q_t, p_s) = H(q_t) + D_{\mathrm{KL}}(q_t \,\|\, p_s).$$

For a fixed teacher target, the entropy term is constant with respect to the student, so minimizing cross-entropy makes the student distribution approach the teacher distribution. The decomposition also clarifies a collapse risk. If the teacher produces the same distribution for every input, the student can match it with a small KL divergence while learning little input-dependent structure. A low loss by itself does not show that the teacher's targets vary meaningfully across examples.

Centering and sharpening address different undesirable target distributions. Without centering, one output coordinate can dominate across many inputs, encouraging the teacher to repeatedly favor the same prototype. Centering counteracts this dominance, but by itself may encourage overly uniform outputs. Sharpening lowers the teacher temperature, making each target distribution more concentrated rather than uniform. Their combination helps retain input-dependent assignments. These are practical anti-collapse mechanisms of the DINO recipe; they should not be generalized into a theorem that centering and sharpening alone guarantee useful features in every architecture or training setup.

Prototype indices are learned output coordinates, not automatically human-defined categories. DINO is best understood as self-distillation with a prototype distribution, rather than as an ordinary clustering algorithm.

Multi-crop training supplies a useful extra constraint. The teacher receives the larger global crops, while the student also sees smaller local crops. A local crop usually lacks some of the global context, so its output must match a target produced from a different field of view. This encourages some features to remain consistent across scales. It does not mean that every local crop contains enough evidence to infer every property of the entire scene; the target and crop protocol define

what consistency is encouraged. The prototype outputs are learned coordinates, not labels assigned by a person, and their semantics must be established through evaluation.

DINOv2 (Oquab et al., 2023) scaled the approach through a curated, diverse image dataset and larger vision transformers, combining image-level self-distillation with masked-patch objectives. DINOv3 (Siméoni et al., 2025) scaled the family further and introduced Gram anchoring to address degradation of dense feature maps during long training. These are training systems, not just alternate equations: data curation, architecture, optimization and feature diagnostics matter alongside the named loss. Their reported results should not be interpreted as a controlled ablation that isolates one change from the rest of the recipe.

3.3.4. Redundancy reduction and explicit variance constraints

BYOL, SimSiam and DINO make training dynamics or teacher targets central to avoiding trivial solutions. Barlow Twins (Zbontar et al., 2021) and VICReg (Bardes et al., 2022) express another idea: require paired views to agree, but constrain the statistics of features across a batch.

3.3.5. Barlow Twins: cross-correlation toward identity

Let Z^A and Z^B be feature batches from two views. Normalize each feature dimension across the batch and define cross-correlation C:Let Z^A and Z^B be feature batches from two views. For dimension i, let μ_i^A and σ_i^A be its mean and standard deviation across the batch, and likewise for view B. Standardize each dimension:

$$\tilde{z}^A_{b,\,i} = \frac{z^A_{b,\,i} - \mu_i^A}{\sigma_i^A + \epsilon}, \qquad \tilde{z}^B_{b,\,i} = \frac{z^B_{b,\,i} - \mu_i^B}{\sigma_i^B + \epsilon}.$$

Then define the cross-correlation matrix C:

$$C_{ij} = \frac{1}{N}\sum_{b=1}^{N} \tilde{z}^A_{b,\,i}\tilde{z}^B_{b,\,j}.$$

If corresponding dimensions agree and distinct dimensions are uncorrelated across views, C approaches the identity matrix. Barlow Twins minimizes the diagonal mismatch and penalizes off-diagonal cross-correlation:

$$L_{\mathrm{BT}} = \sum_i (1 - C_{ii})^2 + \lambda \sum_i \sum_{j \neq i} C_{ij}^2.$$

If C_{ii} is below one, the corresponding coordinate is not perfectly aligned across views. A nonzero C_{ij} for $i \neq j$ means coordinate i in one view is linearly correlated with coordinate j in the other. The loss pushes the cross-correlation matrix toward identity. It controls second-order cross-view statistics; it does not establish statistical independence for arbitrary distributions, nor does it assign

a meaningful physical interpretation to each coordinate.

The diagonal and off-diagonal penalties have different purposes. The diagonal term encourages coordinate i from view A to align with coordinate i from view B. The off-diagonal term discourages coordinate i in one view from correlating with coordinate $j \neq i$ in the other. Barlow Twins does not merely minimize the distance between the two feature vectors: it specifies a pattern for the entire batch cross-correlation matrix.

For intuition, suppose two projected coordinates are duplicates, $z_2 = z_1$. Their off-diagonal cross-correlation is high, so Barlow Twins penalizes that redundancy. If a coordinate does not agree across views, its diagonal correlation falls below one and incurs a diagonal penalty. The objective therefore combines cross-view agreement with pressure against repeated linear information across dimensions. Because the features are standardized over the batch, the correlation matrix describes patterns across examples, not the similarity of two coordinates within a single example.

The standardization is important: the loss is not simply comparing raw feature magnitudes. It measures how two coordinates co-vary across the batch after centering and scaling. A diagonal entry close to one means that a coordinate in the first view tracks its counterpart in the second view; an off-diagonal entry close to zero means that two different coordinates do not exhibit linear cross-view correlation under this batch distribution.

The coefficient λ controls the penalty for off-diagonal correlation relative to diagonal mismatch. If it is small, the objective prioritizes cross-view agreement and permits more redundancy. If it is large, decorrelation receives more weight, but decorrelation is still only a second-order constraint: two coordinates can be uncorrelated while sharing nonlinear dependence. The method reduces one measurable form of redundancy; it does not make each coordinate independent in the full probabilistic sense.

3.3.6. VICReg: make three constraints explicit

VICReg separates invariance, variance and covariance. For two feature batches Z and Z', the invariance term is a mean-squared difference:

$$s(Z, Z') = \frac{1}{N} \| Z - Z' \|_F^2 .$$

For each feature dimension j, let $\sigma_j(Z)$ be its standard deviation over the batch. A variance penalty puts a floor under each dimension's standard deviation:

$$v(Z) = \frac{1}{d} \sum_{j=1}^{d} \max\left(0, \gamma - \sqrt{(Z_j) + \epsilon}\right).$$

The covariance term penalizes the squared off-diagonal entries of the empirical covariance matrix:

$$c(Z) = \frac{1}{d} \sum_{i \neq j} (Z)_{ij}^2.$$

The complete objective combines the three terms:

$$L_{\text{VICReg}} = \lambda s(Z, Z') + \mu [v(Z) + v(Z')] + \nu [c(Z) + c(Z')],$$

where λ, μ, ν set their relative weights. The invariance term aligns paired views. The variance term penalizes a feature dimension whose standard deviation falls below γ. The covariance term discourages linear redundancy between different dimensions. The original VICReg variance term uses an unsquared hinge. Once a feature's standard deviation reaches the margin, that dimension contributes zero variance penalty; the objective enforces a floor rather than rewarding ever-increasing variance.

The sample covariance used for $c(Z)$ is computed after centering the feature columns. For a batch $Z \in \mathbb{R}^{N \times d}$, let $\mu_j = N^{-1} \sum_{b=1}^{N} z_{b,\,j}$. Its covariance entries are

$$(Z)_{ij} = \frac{1}{N-1} \sum_{b=1}^{N} (z_{b,\,i} - \mu_i)(z_{b,\,j} - \mu_j).$$

The variance penalty and covariance penalty are deliberately different. A coordinate can have sufficient variance while duplicating another coordinate; covariance regularization penalizes that redundancy. A coordinate can have low covariance with every other coordinate yet be nearly constant; the variance penalty addresses that case. The invariance term then asks the two views to agree. These terms are complementary, not interchangeable.

The reductions are part of the definition. In $s(Z, Z')$, the squared feature-vector difference is summed over coordinates and averaged over examples; it is not automatically the same scalar as elementwise mean-squared error that divides by both batch size and feature dimension. That difference rescales the invariance term by the embedding dimension, changing the effective balance between λ, μ and ν. When comparing equations and implementations, check whether the norm is summed per example or averaged per coordinate before interpreting the coefficients.

A fully constant representation makes $s(Z, Z') = 0$ and makes all off-diagonal covariances zero, but it also makes each standard deviation zero. For any positive margin γ, the variance term is therefore positive. In this sense, the variance floor makes exact constant collapse incompatible with minimizing all three terms simultaneously. This explicit penalty differs from BYOL or SimSiam, where a constant representation can make the view-agreement part of the objective small. It still does not guarantee that optimization never reaches a bad solution or that nonconstant directions encode the desired semantics.

If every example has the same value in one dimension, its batch standard deviation is zero and the

variance penalty is positive for any positive margin. That makes the anti-collapse constraint explicit. It does not prove that the surviving variation is semantically relevant. A feature can vary with background texture and still satisfy a variance floor.

The methods are related but not identical. Barlow Twins uses normalized cross-correlation as the central matrix objective. VICReg uses a direct invariance term, a standard-deviation floor and a covariance penalty. Their shared lesson is that view agreement and non-collapse are separate requirements. Their moment constraints are not a guarantee of a world-model-ready state.

A simple two-coordinate example makes the distinction concrete. If both coordinates are copies of the same varying scalar, their variances may exceed the margin, so the variance term is satisfied; however, their covariance is high and the covariance term penalizes them. If the second coordinate is constant, its covariance with the first may be zero, but its standard deviation is below the margin and the variance term penalizes it. Only a feature batch with adequate spread and sufficiently low linear redundancy can make both statistical penalties small. The remaining variation can still describe the wrong thing, so a separate task-based evaluation is necessary.

Figure 3.3 compares these mechanisms without implying that they have been trained under a common recipe or that the diagram ranks them.

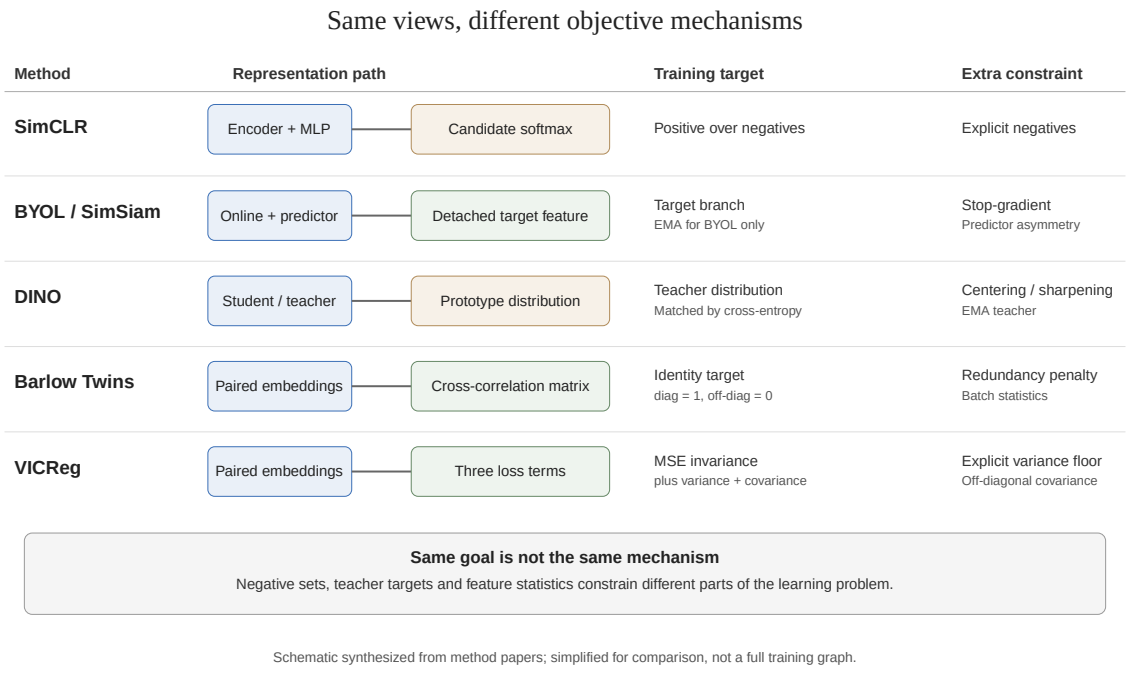


Figure 3.3 Different objectives impose different constraints on the representation

3.3.7. Other methods in brief

DeepCluster alternates between clustering the current feature vectors and training the encoder to predict the resulting cluster assignments. The pseudo-labels therefore change as the representation

changes. **SwAV** computes prototype assignments online and predicts one view's assignment from another view's features; a balancing procedure discourages all examples from being assigned to a small number of prototypes. Both methods turn unlabeled features into pseudo-targets, but differ in how assignments are generated and refreshed.

CLIP (Radford et al., 2021) aligns paired images and text with a contrastive objective; SigLIP (Zhai et al., 2023) replaces the batchwise softmax with pairwise sigmoid losses. Speech systems construct targets from audio itself: wav2vec 2.0 (Baevski et al., 2020) contrasts quantized latent targets, HuBERT (Hsu et al., 2021) predicts cluster-derived labels, and data2vec (Baevski et al., 2022) predicts contextualized teacher features. In language, BERT (Devlin et al., 2019) predicts selected masked tokens from bidirectional context, GPT-style models (Radford et al., 2019) predict the next token from preceding context, and ELECTRA (Clark et al., 2020) predicts whether tokens have been replaced. These examples differ in the target and available context; their full architectures are outside this chapter's main derivations.

Table 3.1 compares the signal each method constructs and the limitation that remains.

Table 3.1 Self-supervised objectives compared by the signal they construct

Method family	**What the objective enforces**	**What it does not guarantee**
SimCLR / MoCo	Identify a positive among sampled candidates	Negatives may be semantically related; features depend on candidate construction
BYOL / SimSiam	Predict a detached view representation	Agreement alone does not rule out constant outputs
DINO family	Match teacher distributions over learned prototypes	Teacher targets can collapse or omit information of interest
Barlow Twins	Cross-view correlation matrix near identity	Second-order decorrelation is not semantic sufficiency
VICReg	View agreement plus variance and covariance constraints	Nonconstant, decorrelated features can still encode nuisance variation
MAE	Predict pixel values at masked patches	Pixel reconstruction does not prove predictive sufficiency
BEiT / iBOT / data2vec	Predict discrete codes or teacher features	The representation inherits the chosen target's biases

Method family	What the objective enforces	What it does not guarantee
DeepCluster / SwAV	Predict or match pseudo-cluster assignments	Cluster assignments need not correspond to downstream concepts

3.3.8. Predicting target representations

Joint-embedding predictive architectures (JEPAs; Assran et al., 2023) predict target-encoder representations from a context representation instead of reconstructing every target pixel or identifying a target among explicit negatives. I-JEPA applies this idea to image regions; video variants extend context and target selection across time. The key design decision is which target regions are predicted from which context. Their detailed objectives belong in the next chapter. Here, they complete the comparison of target choices: agreement, candidate identification, feature statistics, pixels, discrete codes and contextualized representations all impose different constraints.

3.4. Masked prediction: choose what the model must recover

3.4.1. One mask, several target spaces

Suppose an input is divided into visible positions V and masked positions M. An encoder processes the allowed context and a prediction head estimates a target y_i at masked positions:

$$L = \frac{1}{|M|} \sum_{i \in M} \ell(\hat{y}_i, y_i).$$

This form leaves the crucial choice open: what is the target? It might be pixel values, a discrete code, a word token, a replaced-token label or a contextualized feature. The mask tells the model where information is withheld; the target tells it what relationship must be learned.

For a probabilistic predictor, the general objective can be written as a conditional negative log-likelihood:

$$L(\theta) = -\mathbb{E}_{X_V,\, Y_M}[\log p_\theta(Y_M \mid X_V)],$$

where X_V is the visible context and Y_M is the hidden target. If Y_M is a discrete token or code, this becomes cross-entropy. If the model uses a Gaussian likelihood with fixed variance for a continuous target, minimizing negative log-likelihood is equivalent, up to constants and scale, to minimizing squared error. Under squared error, the optimal predictor in expectation is the conditional mean [stem e18d12085ddaca7d965dd899cae3ba0f]]. Under cross-entropy, the optimal predicted class distribution is the conditional distribution of the target given the visible context.

The distinction appears directly in the loss. With squared-error regression, a predictor is rewarded

for numerical proximity to a target vector. With cross-entropy, it is rewarded for assigning probability to the correct discrete label or teacher distribution. These losses define different geometries of error. For a discrete token, predicting a related code is still wrong unless the correct code receives probability; for a continuous target, errors are penalized according to their magnitude in the chosen feature space.

Masking also changes what the model is allowed to observe. A model trained to reconstruct a hidden patch from nearby visible pixels can exploit local texture continuity. A model trained to predict a contextualized teacher feature must reproduce a target shaped by the teacher's wider context and representation space. Neither target is neutral: it specifies which aspects of the missing content distinguish a good prediction from a bad one.

Figure 3.4 holds the mask fixed and changes only the prediction target.

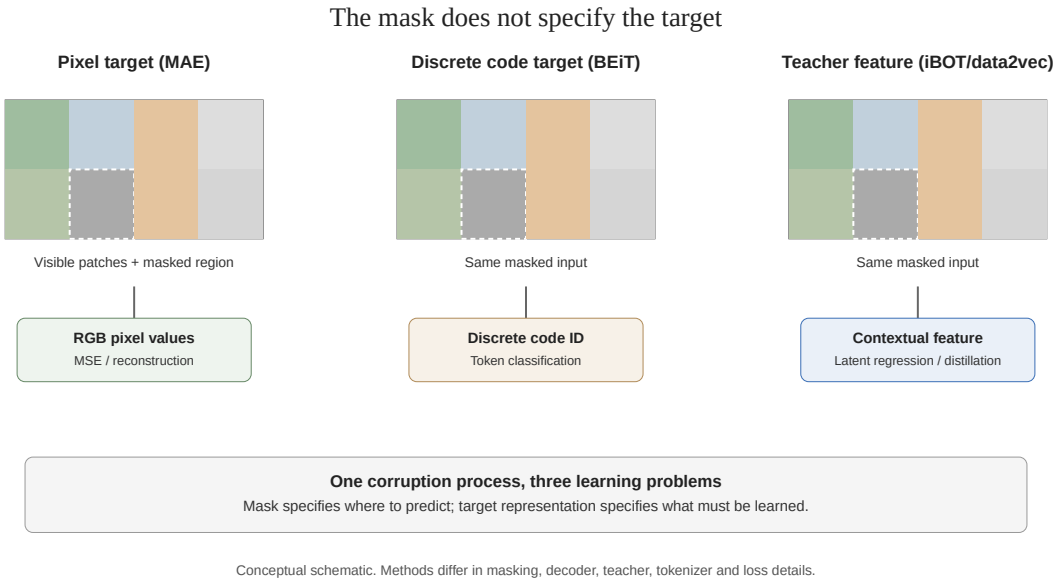


Figure 3.4 The mask is the same; the prediction target changes

3.4.2. MAE: predict masked pixels

A masked autoencoder (MAE; He et al., 2022) divides an image into patches and hides a large fraction of them. Only visible patches pass through the encoder. A smaller decoder receives the encoded visible patches and learned mask tokens inserted at the missing positions; it predicts pixel values at the original patch locations. The asymmetric design saves encoder computation because the encoder never processes the hidden tokens.

Let $m_i = 1$ when patch i is masked and 0 otherwise, with patch target x_i and prediction $\hat{x}_i$. The reconstruction objective is

$$L_{\mathrm{MAE}} = \frac{\sum_i m_i \, \| \, x_i - \hat{x}_i \, \|_2^2}{\sum_i m_i}.$$

The normalization is over masked patches, so visible patches do not contribute to the loss. Under squared error, the conditional mean is the best prediction in expected squared-error terms. If several patch contents are plausible from the visible context, the conditional mean can blur those alternatives. Pixel reconstruction can nevertheless learn useful features; its limitation is that pixel accuracy does not guarantee that the retained encoder feature preserves every variable needed for a later prediction. The encoder, rather than the reconstruction decoder, is usually the component transferred to a downstream task.

The original MAE recipe uses a high masking ratio, 75% of image patches. The encoder therefore processes a sparse visible subset, reducing token-processing cost. The mask ratio also controls prediction difficulty: with few masked patches, more of the image is directly available as context; with too many, the visible content may not determine the missing patches well. The 75% setting is characteristic of the original recipe, not a universal optimum for every resolution, patch size, decoder or target.

Figure 3.5 shows why the MAE encoder sees only visible patches while its decoder reconstructs the masked positions.

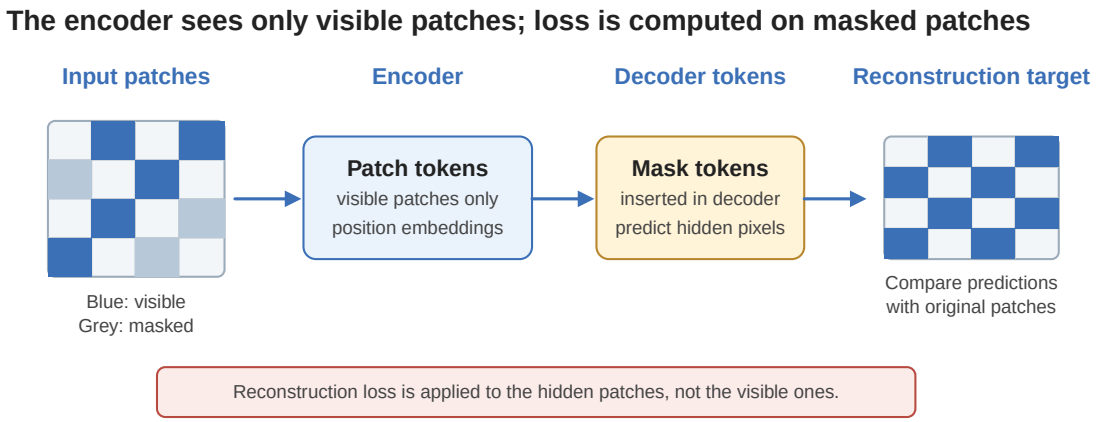


Figure 3.5 MAE encodes visible patches and reconstructs masked patches

3.4.3. BEiT, iBOT and data2vec: change the target

BEiT (Bao et al., 2022) predicts discrete visual-token IDs produced by a learned tokenizer. Rather than regress pixel values, the model classifies the tokenizer's code for each masked patch. The tokenizer introduces its own prior: useful structure can be learned from the code vocabulary, but tokenizer errors and biases enter the target.

iBOT (Zhou et al., 2022) combines masked patch prediction with teacher-student self-distillation.

SimMIM (Xie et al., 2022) uses pixel reconstruction; MaskFeat (Wei et al., 2022) predicts visual features such as HOG; BEiT v2 (Peng et al., 2022) improves the discrete-token route with a semantic visual tokenizer. These methods change the target while keeping the general idea of withholding input content. The target families can be written schematically as

$$L_{\text{BEiT}} = -\log p_\theta(k_i \mid x_{\text{visible}}), \qquad L_{\text{iBOT}} = -\sum_k q_{t,\,i,\,k} \log p_{s,\,i,\,k},$$

where k_i is the tokenizer's discrete code for a masked patch and $q_{t,\,i}$ is a teacher distribution over prototype outputs for patch i. BEiT assigns probability to a discrete target; iBOT matches a teacher distribution. For data2vec, a contextualized teacher vector y_i^t supplies a continuous target and the student predicts that vector under a regression loss such as

$$L_{\text{data2vec},\,i} = \| \hat{y}_i - y_i^t \|_2^2 .$$

These schematic expressions show the target-space distinction; each paper has further implementation choices, including the teacher, normalization, masking protocol and the subset of positions used in its loss. They are not full reproductions of those objectives. The differences reinforce the central point: the mask alone does not define the learning task.

These objectives predict a target **conditioned on what remains visible**. They do not, by themselves, specify a full generative model of the input distribution. A masked predictor may estimate a conditional mean, a discrete target distribution or a teacher feature. It need not define a normalized joint distribution over every possible image or support sampling a complete image. A generative model has additional requirements: its factorization or sampling process must define how a complete observation can be generated. This distinction prevents "reconstructs a patch" from being confused with "learns a generative model of images."

The target also controls the granularity of the error signal. Pixel regression distinguishes exact pixel values and penalizes large numerical deviations more than small ones. Discrete-code cross-entropy cares whether probability is assigned to the correct code, with similarities between codes not automatically reflected in the loss. Teacher-feature regression uses distances in a learned embedding space, so two different target patches may be considered close if the teacher represents them similarly. No target is universally best: the choice decides what counts as a meaningful prediction error.

data2vec (Baevski et al., 2022) predicts contextualized teacher features from masked input and was designed as a framework spanning speech, vision and language. A teacher feature has already aggregated context and training biases from the teacher; it is not simply the original pixel content transformed by a fixed operation. This can make the target more structured, but the student also inherits omissions and biases in the teacher's feature space.

BERT-style masked language modeling predicts selected corrupted tokens from bidirectional

context. GPT-style causal language modeling predicts the next token from preceding tokens only. ELECTRA trains a discriminator to detect replaced tokens. These are brief analogies for target/context construction, not additional implementation tutorials.

3.5. Latent bottlenecks

3.5.1. Dimension is capacity, not useful information

A latent bottleneck maps a high-dimensional observation or token grid into a constrained representation. The simplest form is a vector $h \in \mathbb{R}^d$. Lowering d limits the number of coordinates, but dimension alone does not measure useful information. A small vector may preserve the few variables needed for a prediction; a wider vector may devote many dimensions to texture, illumination or other nuisance variation.

The information-bottleneck principle expresses this trade-off using mutual information. Let X be the observation, Z its representation and Y a target relevant to the later task. A conceptual objective is

$$\min I(X; Z) - \beta I(Z; Y).$$

The first term penalizes information about the input retained by Z; the second rewards information about Y. The coefficient β controls the trade-off. This is a theoretical lens, not the loss explicitly optimized by every method in this chapter. Mutual information for deterministic continuous representations requires care; practical SSL objectives usually optimize a tractable proxy.

The information bottleneck gives a useful idealization of **minimal sufficiency**. If Z is a deterministic function of X and is sufficient for Y, then $Y \perp X \mid Z$, so

$$I(Z; Y) = I(X; Y).$$

The representation has retained all information that X provides about Y, even if it has discarded many details of X unrelated to Y. A minimal sufficient representation seeks to retain that predictive information while minimizing what it carries about the rest of the input. The information-bottleneck objective expresses a trade-off, not a direct recipe for finding a unique vector. Different representations can be equally sufficient, especially when features are transformed by invertible maps.

The important question is not “how small can the vector be?” It is “which distinctions can be discarded without changing the predictions we need to make?” A representation useful for image retrieval may be insufficient for predicting motion if it merges observations with different velocities or hidden states.

This is why raw dimension is not the same as information rate. A real-valued vector with a small number of coordinates can, in principle, encode many distinctions in very fine numerical

differences. In practice, finite precision, noise, limited data and the smoothness of learned functions constrain how reliably such distinctions can be used. Conversely, a very wide feature may carry highly redundant coordinates. Dimension is a proxy for capacity, while useful compression concerns which differences remain reliably decodable under the conditions where the model will be used.

3.5.2. The projection head and the feature that survives

Many contrastive and bootstrap methods use a projection head between an encoder and the objective:

$$h = f_\theta(x), \qquad z = g_\phi(h).$$

The loss is calculated on z, while the representation evaluated downstream is often h. A projection head gives the pretraining objective a dedicated space in which to impose its constraints. It does not follow that z and h have the same geometry or serve the same downstream purpose.

The exported tensor must be identified explicitly. Some evaluations use the feature h before the projector; others use intermediate or patch-level transformer features. A low pretraining loss measured on z is not itself evidence that the feature h has the same geometry or usefulness. The projector can change the geometry substantially because it is a learned, generally nonlinear map. The pretraining loss directly constrains z, not every pairwise relation in h. Claims about representation quality should state the tensor and evaluation protocol rather than infer quality from training loss alone.

3.5.3. A global vector or a grid of local features?

A global vector is compact and convenient for classification, retrieval and simple interfaces. A grid of patch features preserves approximate locations and correspondences. A temporal sequence preserves history only if its aggregation does not erase the changes needed by prediction. These forms are related but not interchangeable.

For patch features $h_1, \ldots, h_P$, simple average pooling forms

$$h_{\text{global}} = \frac{1}{P}\sum_{i=1}^{P} h_i.$$

This compresses the grid into one vector and removes explicit patch indexing from the output. Distinct spatial arrangements can therefore produce the same pooled vector. Retaining the grid preserves a feature for each patch and its token position, although the feature values themselves are not guaranteed to encode accurate geometry. The choice is a representational trade-off: a vector is compact, while a grid makes local correspondences accessible to later components. Global pooling is appropriate when spatial arrangement is irrelevant, but can be harmful when the target depends

on location or correspondence between regions. A temporal representation also has to retain motion-related information when it changes the target future.

The pooling equation also gives a simple source of aliasing. Consider two feature grids whose patch vectors are identical as a multiset but appear in different spatial orders. Average pooling gives the same sum, hence the same global vector, even though the arrangements differ. If the target depends on the arrangement, the pooled representation cannot be sufficient for that target. Positional information elsewhere in the encoder can reduce such collisions, but the pooling operation itself does not preserve the index of each patch.

3.5.4. Compression and predictive sufficiency

A useful representation retains enough information for the relevant prediction while suppressing nuisance variation. There is no universally optimal latent dimension for all tasks. In some settings a small state may be sufficient; in others, spatial features, temporal context or multiple latent components may be necessary.

Here, **compact** means constrained and usable by downstream prediction, not necessarily information-theoretically minimal. The bottleneck should be selected and tested against the downstream task. A low-dimensional representation that aliases important states is not a success; a larger representation is not automatically better if it mostly stores nuisance detail.

Dimension is only a crude capacity indicator. A real-valued vector can encode distinctions in arbitrarily fine numeric differences unless its precision, noise, regularization or decoder is constrained. In practical neural systems, finite precision and limited data make that abstract possibility less useful, but it explains why counting coordinates alone does not measure information. What matters is whether distinctions remain reliably recoverable under the noise and distribution shifts relevant to the downstream problem.

3.6. Compact state representations

A low self-supervised loss establishes only that the representation satisfies its training objective. It does not establish that the representation is useful for the quantity a downstream model must predict. The distinction is mathematical: each pretraining objective constrains a selected relation, whereas predictive sufficiency concerns the full conditional distribution of a specified target.

A linear probe tests whether a selected property is accessible through a linear map:

$$\hat{y} = Wz + b.$$

A poor score does not prove the property is absent; the representation may encode it nonlinearly. A high score establishes linear accessibility under that probe and data split, not sufficiency for every possible target. Probe results should identify the feature tensor, normalization, readout family,

training split and held-out evaluation set. Otherwise, a reported difference can reflect feature selection, readout capacity or data leakage rather than the representation itself.

Other diagnostics measure other things. Per-coordinate variance can detect dimensions that are constant across a batch. The covariance spectrum can reveal whether most feature variation lies in only a few directions. Nearest-neighbor retrieval measures local geometry under a selected distance. Reconstruction evaluates a particular target and decoder. Contrastive retrieval evaluates ranking among a sampled candidate set. These results are not interchangeable: none alone demonstrates that the representation preserves every distinction required to predict the future.

A useful formal definition is **predictive sufficiency**. Let X denote the available observation or observation history, $Z = f(X)$ the representation, and Y the future target. The representation is sufficient for Y if

$$P(Y \mid X) = P(Y \mid Z).$$

Equivalently, $Y \perp X \mid Z$: after conditioning on Z, the full input provides no additional information about Y. This definition is relative to the target distribution. A representation can be sufficient for a coarse category while not being sufficient for a precise location, or sufficient for a one-step target while losing information required for a longer horizon. There is no target-independent scalar score that certifies sufficiency in all these settings.

The failure caused by aliasing follows immediately. Suppose two inputs map to the same representation,

$$f(x_1) = f(x_2) = z, \qquad P(Y \mid x_1) \neq P(Y \mid x_2).$$

A predictor that receives only z must produce the same output distribution for both inputs. It cannot equal both distinct conditional distributions. Increasing the capacity of the downstream predictor cannot restore information removed by the encoder.

Consider two short video histories that end with an object at the same image location. In one history it has been moving left; in the other it has been moving right. If the representation discards the temporal difference and maps both histories to the same z, their next-frame distributions may differ even though the last frames are identical. The example does not imply that velocity must always be explicitly represented as a coordinate. It shows that distinctions that change the target distribution cannot be merged without loss relative to that target.

The practical evaluation should therefore state the claim being tested. A probe supports a claim about accessibility of specified variables. A nearest-neighbor score supports a claim about local feature geometry under a specified metric. A reconstruction metric supports a claim about recovery of a chosen target. A predictive-sufficiency claim is stronger and requires evidence tied to the target distribution and observation history of interest.

Figure 3.6 summarizes the encoder's contract: it should discard nuisance variation without merging observations whose target futures differ.

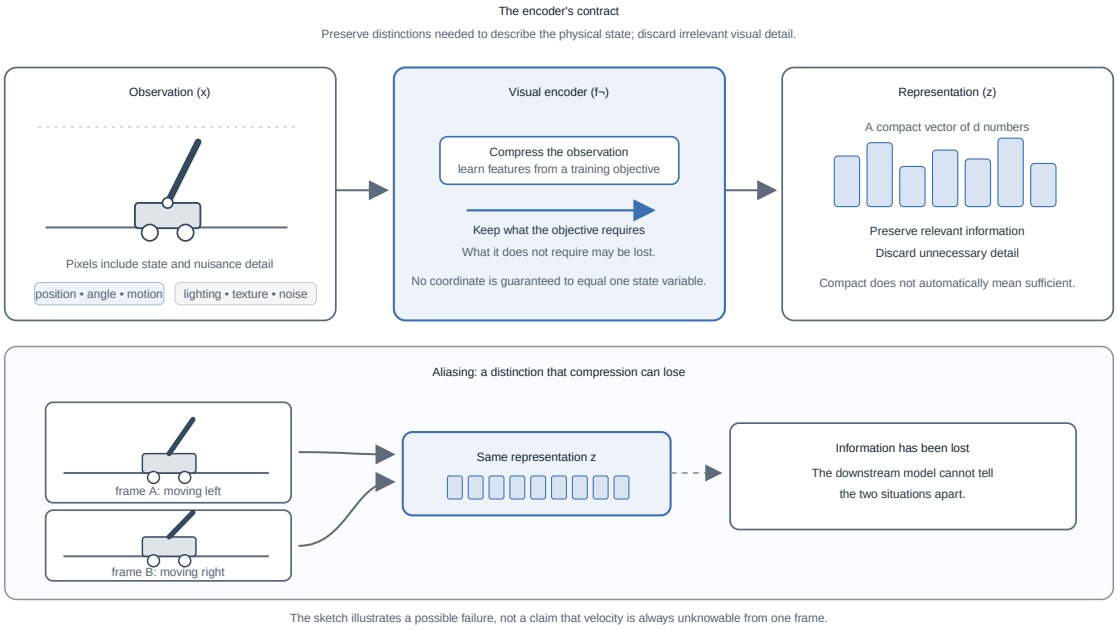


Figure 3.6 The encoder must preserve distinctions relevant to prediction

3.7. Official implementation references

The paper defines the method; the official implementation resolves details that a short equation cannot show, such as the exact normalization, stop-gradient placement, target update, mask convention and training recipe. The repositories below are the primary code references for those details. This chapter does not reproduce their training pipelines.

Method	Official implementation	Inspect for
SimCLR	google-research/simclr	Augmentations, projection head and NT-Xent
MoCo	facebookresearch/moco	Key encoder, queue and momentum update
BYOL	google-deepmind/deepmind-research/byol	Online predictor and EMA target
SimSiam	facebookresearch/simsiam	Shared weights, predictor and stop-gradient
DINO / DINOv2 / DINOv3	DINO; DINOv2; DINOv3	Teacher targets, masking and family-specific training details

Method	Official implementation	Inspect for
Barlow Twins	facebookresearch/barlowtwins	Batch normalization and cross-correlation loss
VICReg	facebookresearch/vicreg	Variance floor, covariance and loss scaling
MAE	facebookresearch/mae	Visible-token encoder, decoder and masked-only loss
DeepCluster / SwAV	DeepCluster; SwAV	Pseudo-labels, prototypes and balanced assignments
CLIP	openai/CLIP	Image-text pairing and bidirectional contrastive loss

Use the paper to understand the mathematical objective and the official source to confirm how that objective is implemented. Code paths change between releases, so implementation-specific statements should be tied to a repository revision rather than inferred from a method name alone.

3.8. Summary

Self-supervised learning constructs a training target from data. Contrastive methods identify a positive among sampled candidates; their gradients depend on the candidate distribution, and their information-theoretic interpretation requires specific sampling assumptions. BYOL and SimSiam predict a detached feature target but differ in whether that target is updated by EMA. DINO matches teacher output distributions. Barlow Twins penalizes departures of cross-view correlation from identity, while VICReg separates view agreement, a variance floor and covariance regularization.

Masked prediction changes both the hidden context and the target: MAE predicts pixels, BEiT predicts discrete codes, and iBOT/data2vec use teacher-derived targets. The choice of target determines what errors are penalized. The latent bottleneck then determines which features are retained and which distinctions may be lost. A useful representation must preserve enough information for the prediction of interest; the pretraining loss by itself cannot establish that condition.

3.9. Exercises

1. For an anchor with one positive and $K-1$ negatives, derive the softmax-cross-entropy gradient with respect to each scaled similarity logit. Explain why a high-probability negative receives a

larger gradient than a low-probability negative.

2. Using cosine similarities 0.80, 0.10, and -0.30 with temperature 0.5, calculate the three softmax probabilities and the loss assigned to the positive.
3. State the parameter-update difference between BYOL and SimSiam. In each method, which pathway receives gradients, and which mechanism changes the target?
4. Write the Barlow Twins objective in terms of the cross-correlation matrix. Explain the role of its diagonal and off-diagonal terms, then contrast it with VICReg's three terms.
5. Compare the targets used by MAE, BEiT and iBOT/data2vec. For each method, state what is predicted and what the loss measures.
6. Give two observation histories x_1, x_2 that map to the same representation but have different conditional future distributions. Explain why no downstream predictor that receives only that representation can predict both distributions correctly.
7. Decompose DINO's teacher-student cross-entropy into target entropy and KL divergence. Explain what happens if the teacher returns the same distribution for every image.
8. For a batch whose second VICReg coordinate is constant but whose first coordinate varies, determine which term penalizes the second coordinate and why the covariance penalty alone would not be enough.

3.10. References

Assran, M. et al. (2023). Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture. **IEEE/CVF Conference on Computer Vision and Pattern Recognition**. arXiv:2301.08243.

Baevski, A., Zhou, Y., Mohamed, A. and Auli, M. (2020). wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. **Advances in Neural Information Processing Systems**. arXiv:2006.11477.

Baevski, A., Hsu, W.-N., Xu, Q., Babu, A., Gu, J. and Auli, M. (2022). data2vec: A General Framework for Self-Supervised Learning in Speech, Vision and Language. **International Conference on Machine Learning**. arXiv:2202.03555.

Bardes, A., Ponce, J. and LeCun, Y. (2022). VICReg: Variance-Invariance-Covariance Regularization For Self-Supervised Learning. **International Conference on Learning Representations**. arXiv:2105.04906.

Bardes, A. et al. (2024). Revisiting Feature Prediction for Learning Visual Representations from Video. arXiv:2404.08471.

Bao, H., Dong, L. and Wei, F. (2022). BEiT: BERT Pre-Training of Image Transformers.

International Conference on Learning Representations. arXiv:2106.08254.

Caron, M. et al. (2018). Deep Clustering for Unsupervised Learning of Visual Features. **European Conference on Computer Vision**. arXiv:1807.05520.

Caron, M. et al. (2020). Unsupervised Learning of Visual Features by Contrasting Cluster Assignments. **Advances in Neural Information Processing Systems**. arXiv:2006.09882.

Caron, M. et al. (2021). Emerging Properties in Self-Supervised Vision Transformers. **IEEE/CVF International Conference on Computer Vision**. arXiv:2104.14294.

Chen, T. et al. (2020). A Simple Framework for Contrastive Learning of Visual Representations. **International Conference on Machine Learning**. arXiv:2002.05709.

Chen, X. and He, K. (2021). Exploring Simple Siamese Representation Learning. **IEEE/CVF Conference on Computer Vision and Pattern Recognition**. arXiv:2011.10566.

Chen, X. et al. (2020). Improved Baselines with Momentum Contrastive Learning. arXiv:2003.04297.

Chen, X. et al. (2021). An Empirical Study of Training Self-Supervised Vision Transformers. arXiv:2104.02057.

Clark, K. et al. (2020). ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators. **International Conference on Learning Representations**. arXiv:2003.10555.

Devlin, J. et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. **North American Chapter of the Association for Computational Linguistics**. arXiv:1810.04805.

Doersch, C., Gupta, A. and Efros, A. A. (2015). Unsupervised Visual Representation Learning by Context Prediction. **IEEE International Conference on Computer Vision**. arXiv:1505.05192.

Gidaris, S., Singh, P. and Komodakis, N. (2018). Unsupervised Representation Learning by Predicting Image Rotations. **International Conference on Learning Representations**. arXiv:1803.07728.

Grill, J.-B. et al. (2020). Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning. **Advances in Neural Information Processing Systems**. arXiv:2006.07733.

He, K. et al. (2020). Momentum Contrast for Unsupervised Visual Representation Learning. **IEEE/CVF Conference on Computer Vision and Pattern Recognition**. arXiv:1911.05722.

He, K. et al. (2022). Masked Autoencoders Are Scalable Vision Learners. **IEEE/CVF Conference on Computer Vision and Pattern Recognition**. arXiv:2111.06377.

Hsu, W.-N. et al. (2021). HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units. arXiv:2106.07447.

Noroozi, M. and Favaro, P. (2016). Unsupervised Learning of Visual Representations by Solving Jigsaw Puzzles. **European Conference on Computer Vision**. arXiv:1603.09246.

Oquab, M. et al. (2023). DINOv2: Learning Robust Visual Features without Supervision. arXiv:2304.07193.

Peng, Z. et al. (2022). BEiT v2: Masked Image Modeling with Vector-Quantized Visual Tokenizers. arXiv:2208.06366.

Radford, A. et al. (2019). Language Models are Unsupervised Multitask Learners. OpenAI technical report.

Radford, A. et al. (2021). Learning Transferable Visual Models From Natural Language Supervision. **International Conference on Machine Learning**. arXiv:2103.00020.

Siméoni, O. et al. (2025). DINOv3. arXiv:2508.10104.

van den Oord, A., Li, Y. and Vinyals, O. (2018). Representation Learning with Contrastive Predictive Coding. arXiv:1807.03748.

Wei, C. et al. (2022). Masked Feature Prediction for Self-Supervised Visual Pre-Training. **IEEE/CVF Conference on Computer Vision and Pattern Recognition**. arXiv:2112.09133.

Xie, Z. et al. (2022). SimMIM: A Simple Framework for Masked Image Modeling. **IEEE/CVF Conference on Computer Vision and Pattern Recognition**. arXiv:2111.09886.

Zhai, X. et al. (2023). Sigmoid Loss for Language Image Pre-Training. **IEEE/CVF International Conference on Computer Vision**. arXiv:2303.15343.

Zbontar, J. et al. (2021). Barlow Twins: Self-Supervised Learning via Redundancy Reduction. **International Conference on Machine Learning**. arXiv:2103.03230.

Zhang, R., Isola, P. and Efros, A. A. (2016). Colorful Image Colorization. **European Conference on Computer Vision**. arXiv:1603.08511.

Zhou, J. et al. (2022). iBOT: Image BERT Pre-Training with Online Tokenizer. arXiv:2111.07832.